

HLD / System Design

The memory map. Every building block with one desi analogy, so the whiteboard round stops feeling like a black box. 0 to interview-ready.

[@rathoreadiitya](#) - Instagram

v1.0 · Updated 2026-06

Inside this sheet

1. Start here: the student minimum - what an HLD round ACTUALLY tests
2. Why system design feels overwhelming - and the fix
3. The request lifecycle - how one click reaches a server and back
4. The core tradeoffs in one screen - the vocabulary you must speak
5. THE BUILDING-BLOCK MEMORY MAP - all 15 at a glance (screenshot this)
6. Beyond the basics - how to read the deep dives
7. The 15 building blocks - one analogy each (deep dive)
8. The confusion-killer - the pairs people always mix up
9. Back-of-the-envelope numbers every fresher must know
10. The framework - attack ANY system design problem in 7 steps
11. Interview triggers - if they say X, reach for Y
12. Common mistakes in HLD rounds - what loses points
13. Communication & APIs - how clients and services talk
14. Interview FAQs - the questions every panel circles back to
15. More concepts to recognize - know the word, say the one-liner
16. Revision - the last 20 minutes before the round

If you have only 20 minutes -> jump to section 16 (Revision), the memory map in section 5, and the framework in section 10.

1. Start here: the student minimum

Honest truth first. A fresher / SDE-1 system design round is NOT asking you to design Google from scratch. Most interviewers want to see that you can take a vague problem, draw a sane high-level diagram, and reason about a few trade-offs out loud. What actually gets tested:

1. The request flow - can you draw client -> load balancer -> servers -> cache -> database and explain each box.
2. A handful of building blocks: load balancer, caching, SQL vs NoSQL, replication, sharding, and a message queue. Know what each does and WHEN to reach for it.
3. Trade-off talk - SQL vs NoSQL, strong vs eventual consistency, scale up vs out. You don't need the 'right' answer, you need to weigh both sides.

4. Designing ONE system end to end out loud. A URL shortener or a rate limiter is the classic warm-up; Twitter feed / WhatsApp are the step-up.

What a fresher HLD round ACTUALLY tests

Nobody asks you to design Google from scratch. They test these four things.

1 Draw the request flow
Client → load balancer → servers → cache → database, and explain each box.

2 Know a few building blocks
Load balancer, caching, SQL vs NoSQL, replication, sharding, message queue - what each does and WHEN.

3 Talk trade-offs
SQL vs NoSQL, strong vs eventual, scale up vs out. Weigh both sides out loud.

4 Design ONE system end to end
URL shortener or rate limiter is the warm-up; Twitter feed / WhatsApp is the step-up.

The one hands-on thing: design a URL shortener fully on paper - API, the hashing, the DB choice, where a cache fits. Walking through ONE design beats memorizing twenty definitions.

The four things a fresher round actually checks. Nobody asks you to design Google from scratch.

The one hands-on thing worth doing: design a URL shortener end to end on paper - API, the hashing, the database choice, and where a cache fits. Walking through ONE design beats memorizing twenty definitions you can't apply.

Time-pressed? Read the memory map (section 5), the framework (section 10), and the revision page (section 14). Treat the deeper blocks as recognize-and-reason, not memorize-cold.

2. Why system design feels overwhelming

DSA has a compile button: your code passes or it doesn't. System design has an open whiteboard and an interviewer saying 'design Instagram'. Most students collect a pile of scary words (sharding, CAP, eventual consistency) that never became pictures they can use.

The fix is the same one that works for design patterns: ONE physical picture per concept. You won't remember 'consistent hashing minimizes remapping on resize'. You WILL remember a round table where adding one guest barely disturbs the others. Learn the picture first, the definition second, the trade-off third.

Why system design feels overwhelming - and the fix

DSA has a compile button. System design has an open whiteboard. The fix: one picture per concept.

THE DEFINITION (FORGOTTEN UNDER PRESSURE)

"Consistent hashing minimizes key remapping when the cluster is resized."



THE PICTURE (SURVIVES THE INTERVIEW)

A round table - add one guest and the others barely shift.

THE DEFINITION

"Eventual consistency: replicas converge after a bounded delay."



THE PICTURE

A WhatsApp like count - off by 2 for a second, nobody is harmed.

THE DEFINITION

"A load balancer distributes requests across a server pool."



THE PICTURE

Station ticket windows - one guy splits the crowd across counters.

Learn the picture first, the definition second, the trade-off third. That order is how you sound like an engineer, not a flashcard.

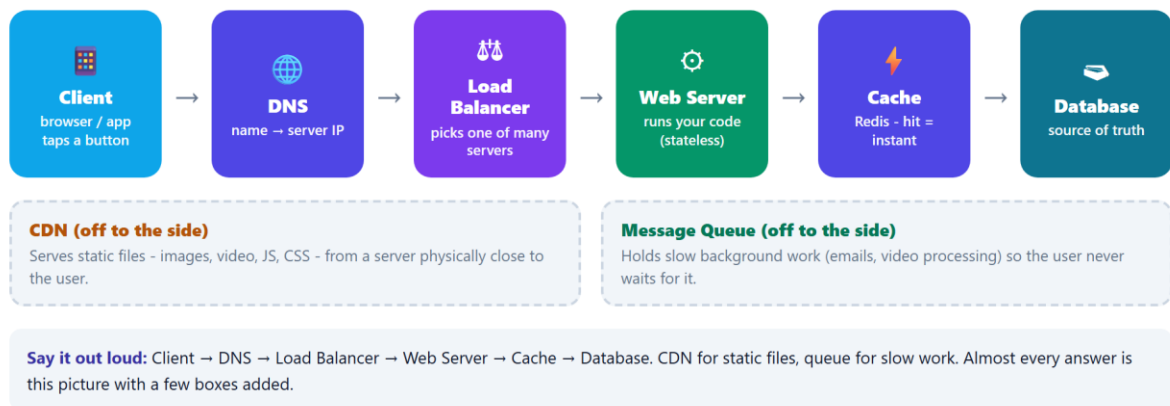
The definition is forgotten under pressure. The picture survives.

3. The request lifecycle - how one click reaches a server

Before any building block makes sense, you need the one diagram every design is built on: what happens between a user tapping a button and seeing a result. If you can draw and narrate this, you can start ANY system design answer.

The request lifecycle - how one click reaches a server

Every system design diagram is this skeleton. Learn to draw and narrate it, then add boxes where the load is.



The path of a single request. Every HLD diagram is this skeleton, with boxes added or split as the system grows.

5. DNS - your browser turns 'instagram.com' into a server IP address (like a phonebook lookup: name -> number).

6. Load balancer - the request hits a load balancer, which picks one of many identical web servers to handle it.
7. Web / app server - runs your code (the business logic). It is kept stateless so any server can handle any request.
8. Cache - the server first checks a fast cache (Redis). A hit returns instantly; a miss falls through to the database.
9. Database - the source of truth. Reads may come from replicas; writes go to the primary.
10. Response travels back the same way. Static files (images, video) are served separately by a CDN close to the user.

Memorize this spine: Client -> DNS -> Load Balancer -> Web Server -> Cache -> Database, with a CDN off to the side for static files and a message queue off to the side for slow background work. Almost every system design answer is this picture with a few boxes added.

4. The core tradeoffs in one screen

System design is a trade-off conversation. These are the words you must be able to use correctly. Interviewers listen for exactly this vocabulary.

The core tradeoffs - the vocabulary you must speak

System design is a trade-off conversation. Name what you give up, not just what you pick.

Latency time for ONE request - optimize for user-facing actions	vs	Throughput requests per second - optimize for pipelines & batch jobs
Consistency show only the latest data - money, inventory, seats	vs	Availability always answer, maybe stale - feeds, likes, DNS
Scale up one bigger machine - easy, but has a ceiling + SPOF	vs	Scale out many machines - near-unlimited, needs stateless + LB
Read-heavy far more reads - reach for caching + replicas	vs	Write-heavy lots of writes - reach for sharding + queues

The meta-rule: there is no free lunch. 'I'll use a cache, trading a small risk of stale data for much lower latency' scores higher than just 'I'll add a cache'.

Every choice buys one thing by giving up another. Name what you trade away.

Trade-off	Plain-English meaning	The rule of thumb
Latency vs Throughput	Latency = time for ONE request. Throughput = how MANY requests per second.	Optimize latency for user-facing actions; optimize throughput for pipelines and batch jobs.

Trade-off	Plain-English meaning	The rule of thumb
Consistency vs Availability	Show only the latest data vs always answer (maybe with stale data).	Money / inventory -> consistency. Feeds / likes / DNS -> availability.
Scale up vs Scale out	One bigger machine vs many smaller machines.	Scale up first (easy), scale out when one box can't cope or you need fault tolerance.
Stateful vs Stateless	Server remembers the user's session vs server remembers nothing between requests.	Keep web servers stateless so a load balancer can send any request to any server.
Read-heavy vs Write-heavy	Far more reads than writes vs lots of writes.	Read-heavy -> caching + replicas. Write-heavy -> sharding + message queues.
SQL vs NoSQL	Structured + transactions vs flexible + scales out.	Relationships + correctness -> SQL. Massive scale + flexible schema -> NoSQL.

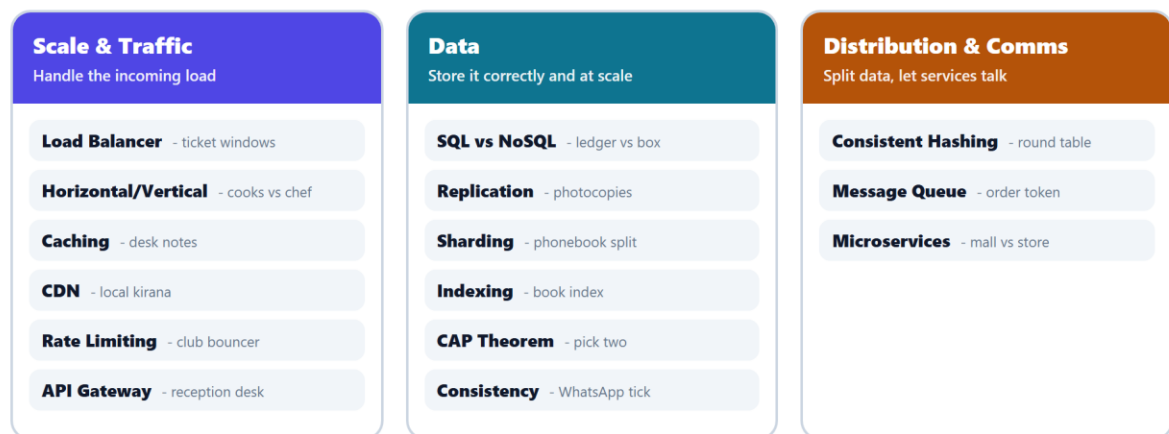
The meta-rule behind all of these: there is no free lunch. Every choice buys one thing by giving up another. Interviewers don't want the 'correct' pick, they want you to NAME what you're trading away. 'I'll use a cache, trading a small risk of stale data for much lower latency' scores higher than just 'I'll add a cache'.

5. The building-block memory map - all 15 at a glance

This is the table to screenshot. The hook column is your one-line recall trigger. Learn the hook first, the formal definition second.

The 15 building blocks, in 3 layers

Scale & Traffic handles the load. Data stores it right. Distribution & Comms splits it and lets services talk.



One-line rule: 'balance / scale / cache' = Scale & Traffic. 'store / copy / split / consistent' = Data. 'distribute / async / split services' = Distribution & Comms.

The 3 layers. Scale & Traffic = handle the load. Data = store it right. Distribution & Comms = split it and let services talk.

#	Building block	Layer	One line to remember	What it does
1	Load Balancer	Scale & Traffic	Bheed ko counters pe baant do. (railway ticket windows)	Spreads incoming requests across many identical servers so no single one gets overwhelmed, and routes around servers that die.
2	Horizontal vs Vertical Scaling	Scale & Traffic	Ek super-chef vs zyada cooks. (kitchen)	Vertical = make one server bigger (more CPU/RAM). Horizontal = add more servers and split the load across them.
3	Caching	Scale & Traffic	Baar-baar wali cheez haath ke paas rakho. (desk pe notes)	Stores a copy of frequently-read data in fast memory (RAM) so most reads skip the slow database.
4	CDN	Scale & Traffic	Door ki factory nahi, paas ki dukaan. (local kirana)	A worldwide network of edge servers that store copies of your static content close to users, so files load fast no matter where they are.
5	Rate Limiting	Scale & Traffic	Bouncer entry ginta hai. (club gate)	Caps how many requests a user/IP can make in a time window, protecting the system from abuse and overload.
6	API Gateway	Scale & Traffic	Building ka reception desk. (visitor routing)	A single entry point in front of many backend services that handles auth, routing, rate limiting, and logging in one place.
7	SQL vs NoSQL	Data	Pakka ledger vs Flexible sticky-note box. (accounts)	SQL = structured tables with a fixed schema and strong consistency. NoSQL = flexible documents/key-values that scale out easily.
8	Database Replication	Data	Ledger ki photocopy, sab padh sakein. (backup copies)	Keeps multiple copies of the database in sync so reads can be spread across copies and the system survives a server dying.
9	Sharding / Partitioning	Data	Ek mota phonebook ko A-M, N-Z mein todo. (directory split)	Splits one huge dataset across many machines by a shard key, so each machine holds and serves only a slice.
10	Database Indexing	Data	Kitaab ke peeche ka index. (page dhoondho)	A sorted lookup structure on a column that turns a slow full-table scan into a fast direct jump to the rows you want.
11	CAP Theorem	Data	Teen mein se do hi mil sakte hain. (pick two)	In a distributed system, when the network between nodes breaks, you must choose between staying Consistent (refuse stale answers) or staying Available (answer with possibly-stale data).

#	Building block	Layer	One line to remember	What it does
1 2	Consistency Models	Data	Turant pakka vs thodi der mein pakka. (WhatsApp tick)	Strong = every read sees the latest write, always. Eventual = reads may briefly see old data, but all copies converge soon.
1 3	Consistent Hashing	Distribution & Comms	Gol mez pe seat, naya mehmaan sabko na hilaaye. (round table)	A hashing scheme that lets you add or remove servers while moving only a small fraction of the data, instead of remapping everything.
1 4	Message Queue	Distribution & Comms	Order token lo, kitchen apne pace pe banaye. (Swiggy queue)	A buffer between services that lets a producer drop a task and move on, while a consumer processes it later, decoupling them.
1 5	Microservices vs Monolith	Distribution & Comms	Ek bada general store vs specialty shops ka mall. (architecture)	Monolith = one codebase/deployable doing everything. Microservices = many small independent services, each owning one job and deployed separately.

Remember the 3 layers in one line: SCALE & TRAFFIC handles the incoming load (balancer, scaling, cache, CDN, rate limit, gateway). DATA stores it right (SQL/NoSQL, replication, sharding, indexing, CAP, consistency). DISTRIBUTION & COMMS splits the data and lets services talk (consistent hashing, message queue, microservices).

6. Beyond the basics (deeper than a fresher round needs)

For a fresher / SDE-1 round, the must-know core is: load balancer, horizontal scaling, caching, SQL vs NoSQL, replication, sharding, and message queues. Master those seven well enough to draw and defend them. Treat CAP, consistency models, consistent hashing, rate limiting, API gateway, indexing, CDN, and microservices as recognize-and-reason: know the analogy and when each fits, but you'll rarely design one cold as a fresher. Go deeper only for SDE-2 and senior rounds.

7. The 15 building blocks - one analogy each

1. Load Balancer (Scale & Traffic)

Analogy: Station pe ek hi ticket window ho toh line lambi ho jaati hai. Isliye 10 windows khol dete hain, aur ek banda bheed ko har window pe baant deta hai. Load balancer wahi banda hai: aane wali requests ko kai servers pe baant deta hai taaki koi ek server overload na ho.

What it does: Spreads incoming requests across many identical servers so no single one gets overwhelmed, and routes around servers that die.

Spot it when you hear:

- 'Distribute traffic across multiple servers'
- 'What happens if one server goes down?'
- 'How do you avoid a single point of failure at the web tier?'

How it works

```
Client -> [ Load Balancer ] -> Server 1
                                -> Server 2   (round-robin / least-connections)
                                -> Server 3
Server 2 stops replying -> health check fails -> LB stops sending it traffic
```

Trap: People say 'load balancer' and stop. Interviewers push: WHICH algorithm? Round-robin (simple, equal turns), least-connections (send to the least busy), IP-hash (same user lands on the same server). And L4 vs L7: L4 routes on IP/port (fast, dumb), L7 reads the actual HTTP request so it can route by URL or cookie (smarter, slightly slower).

Real use: Sits in front of the web tier in almost every design. Nginx, HAProxy, AWS ELB / ALB.

Worth knowing: The load balancer itself can be a single point of failure. The fix: run two of them in active-passive, so if the primary dies the standby takes over.

Don't confuse: Load balancer SPREADS traffic across many servers. A reverse proxy SITS in front of servers to cache, compress, or hide them (one LB is usually also a reverse proxy, but the job is different).

2. Horizontal vs Vertical Scaling (Scale & Traffic)

Analogy: Restaurant mein order badh gaye. Do raaste: ek hi chef ko aur tez, aur mehnati banao (vertical, bade machine), ya kitchen mein 5 aur cooks bharti karo (horizontal, zyada machines). Ek chef ki limit aati hai. Zyada cooks lagbhag infinitely badhaye ja sakte hain, par unhe coordinate karna padta hai.

What it does: Vertical = make one server bigger (more CPU/RAM). Horizontal = add more servers and split the load across them.

Spot it when you hear:

- 'How will this handle 10x the traffic?'
- 'Scale up or scale out?'
- 'What is the limit of a single machine?'

How it works

VERTICAL (scale up): 1 server -> bigger server (2x CPU, 2x RAM)
simple, no code change, but a hard ceiling + single point of failure

HORIZONTAL (scale out): 1 server -> many servers behind a load balancer
near-unlimited, fault-tolerant, but needs stateless servers + a LB

Trap: Horizontal scaling only works if your servers are **STATELESS**, meaning they store no user session in local memory. If user data lives on Server 1, the load balancer can't freely send the next request to Server 2. **Fix:** push session/state into a shared store (Redis) or a database, so any server can handle any request.

Real use: Every 'scale from zero to millions' story starts vertical (cheap, easy) then goes horizontal once one box isn't enough.

Don't confuse: Vertical = ONE machine gets bigger (ceiling + SPOF). Horizontal = MORE machines (needs stateless + a load balancer). Real systems do horizontal for the web tier.

3. Caching (Scale & Traffic)

Analogy: Exam ke time tum important formulae library se baar-baar nahi laate, ek chit pe likh ke desk pe rakh lete ho. Library door hai (database), chit paas hai (cache). Cache database ke saamne ek tez, chhoti memory hai jisme sabse zyada maange jaane wale answers rakhe jaate

hain, taaki har baar database tak na jaana pade.

What it does: Stores a copy of frequently-read data in fast memory (RAM) so most reads skip the slow database.

Spot it when you hear:

- 'This read is slow / the database is the bottleneck'
- 'The same data is requested again and again'
- 'Reduce latency and database load'

How it works

Read request -> is it in cache?

HIT -> return from cache (microseconds, no DB hit)

MISS -> read DB -> put a copy in cache -> return (this is cache-aside)

Eviction when cache is full: LRU (drop least-recently-used) is the default.

Caching - hit vs miss (cache-aside)

A fast copy in RAM in front of the slow database. Most reads should never touch the DB.

Cache HIT

Read request → data is **in cache** → return from RAM.

Microseconds. The database is never touched. This is what a good cache hit ratio (80%+) buys you.

Cache MISS

Read request → **not in cache** → read the database → put a copy in cache → return.

Slow this once, fast every time after - until it's evicted (LRU) or expires (TTL).

The hard part: stale data. The DB updated but the cache still holds the old value. Fix with a TTL (auto-expire) or invalidate the key on every write.

Trap: Stale data. The DB updated but the cache still holds the old value. Two fixes: set a TTL (the entry auto-expires after N seconds), or invalidate the cache key on every write. 'There are only two hard things in CS: cache invalidation and naming things' is a real interview joke, and a real problem.

Real use: Redis / Memcached in front of the DB. Also browser cache, CDN, and DB query cache. A good cache hit ratio (say 80%+) cuts DB load massively.

Worth knowing: Write strategies worth naming: write-through (write to cache AND DB together, safe but slower), write-back (write to cache first, DB later, fast but risky if cache dies), cache-aside (the app manages it, the most common one).

Don't confuse: A cache is a fast copy of data you OWN (in your RAM). A CDN is a cache of

static files placed physically CLOSE to the user. Same idea (store a copy), different goal (speed vs distance).

4. CDN (Scale & Traffic)

Analogy: Tumhe Maggi chahiye toh factory tak nahi jaate, paas ki kirana dukaan se le aate ho. CDN aisi hi kirana dukaanon ka network hai duniya bhar mein. Tumhari website ki images, video, CSS in dukaanon pe pehle se rakhi hoti hain, toh user ko file paas ke server se milti hai, na ki tumhare door wale main server se.

What it does: A worldwide network of edge servers that store copies of your static content close to users, so files load fast no matter where they are.

Spot it when you hear:

- 'Users are spread across the globe and assets load slowly'
- 'Serve images / video / JS / CSS fast'
- 'Reduce load on the origin server'

How it works

User in India -> nearest CDN edge (Mumbai) -> file already cached -> fast
Edge doesn't have it -> pulls once from origin -> caches it -> serves all future users

Origin = your main server. Edge = the CDN copy near the user.

Trap: CDNs are for STATIC content (images, video, fonts, JS bundles), not for personalized or fast-changing data. You wouldn't cache a user's live bank balance on a CDN. Also, when you update a file, the old copy can linger at the edge until its TTL expires, so you 'bust the cache' by versioning the filename (app.v2.js).

Real use: Cloudflare, Akamai, AWS CloudFront. Any app with images/video and a global audience uses one.

Don't confuse: A CDN caches by physical DISTANCE (closer = faster). A regular cache (Redis) caches by ACCESS SPEED (RAM vs disk). CDN serves static files; Redis serves any data your app needs fast.

5. Rate Limiting (Scale & Traffic)

Analogy: Club ka bouncer ek minute mein utne hi log andar jaane deta hai. Zyada aaye toh 'ruko, baad mein'. Rate limiter wahi bouncer hai: ek user ya IP ko ek window mein utni hi requests bhejne deta hai. Isse ek banda spam karke poora system gira nahi sakta.

What it does: Caps how many requests a user/IP can make in a time window, protecting the system from abuse and overload.

Spot it when you hear:

- 'Stop one user from spamming the API'
- 'Protect against abuse / DDoS / scrapers'
- 'Allow only N requests per minute'

How it works

Request -> rate limiter: has this user used up their quota this window?
under limit -> allow, count it
over limit -> reject with HTTP 429 Too Many Requests

Common algorithms: token bucket, leaky bucket, fixed/sliding window.

Trap: Where do you store the counts? In a single server's memory it breaks the moment you scale horizontally (each server has its own count). **Fix:** keep the counters in a shared store like Redis, so all servers enforce one combined limit per user.

Real use: Every public API (login attempts, payment retries, search). Token bucket is the most common algorithm because it allows short bursts.

Don't confuse: Rate limiting protects you from TOO MANY requests (abuse). A load balancer SPREADS the requests you accept. One says 'no, slow down'; the other says 'okay, you go to server 3'.

6. API Gateway (Scale & Traffic)

Analogy: Badi company ki building mein ek reception hota hai. Visitor seedhe kisi bhi floor pe

nahi ghus sakta. Reception ID check karta hai, batata hai kaunse floor jaana hai, aur entry log karta hai. API gateway wahi reception hai: saari client requests ek hi door se aati hain, aur gateway unhe sahi backend service tak bhejta hai.

What it does: A single entry point in front of many backend services that handles auth, routing, rate limiting, and logging in one place.

Spot it when you hear:

- 'You have many microservices, how do clients talk to them?'
- 'Where do auth and rate limiting live?'
- 'One front door for the whole backend'

How it works

```
Client -> [ API Gateway ] -> Auth service
                        does: auth,      -> Orders service
                        routing,         -> Payments service
                        rate limiting    -> Search service
```

One door, cross-cutting concerns handled once instead of in every service.

Trap: The gateway can become a bottleneck and a single point of failure, since all traffic flows through it. Run multiple instances behind a load balancer. Also don't stuff business logic into the gateway, keep it to cross-cutting concerns (auth, routing, throttling), or it turns into a mini-monolith.

Real use: Microservice architectures. AWS API Gateway, Kong, Nginx. Often paired with a load balancer.

Don't confuse: An API gateway is the smart FRONT DOOR for the whole backend (auth + routing + limits). A load balancer just SPREADS traffic across copies of one service. A gateway usually has a load balancer behind it.

7. SQL vs NoSQL (Data)

Analogy: SQL ek pakke ruled ledger jaisa hai: har entry ka fixed format, columns fixed, rishte (relationships) saaf. NoSQL ek dabba hai jisme alag-alag shape ki sticky-notes daal sakte ho, format jab chahe badal lo. Ledger discipline deta hai (banking), dabba flexibility aur scale deta

hai (feeds, logs).

What it does: SQL = structured tables with a fixed schema and strong consistency. NoSQL = flexible documents/key-values that scale out easily.

Spot it when you hear:

- 'What database would you pick and why?'
- 'The data is unstructured / the schema changes a lot'
- 'Do you need transactions or massive scale?'

How it works

SQL (Postgres, MySQL): rows + columns, JOINS, ACID transactions
pick when: money, relationships, you need strong consistency

NoSQL (MongoDB, Cassandra, DynamoDB): documents / key-value / wide-column
pick when: huge scale, flexible schema, mostly simple lookups

Trap: 'NoSQL is always faster / always scales better' is wrong. NoSQL scales writes by giving up easy JOINS and (often) strong consistency. If your data is relational and you need transactions (a bank transfer), forcing NoSQL makes life harder. Pick by the access pattern, not by hype.

Real use: Banking, e-commerce orders -> SQL. Activity feeds, IoT/logs, chat history, catalogs -> NoSQL. Many real systems use both.

Worth knowing: The 4 NoSQL types: key-value (Redis, DynamoDB), document (MongoDB), wide-column (Cassandra), graph (Neo4j). Know one example of each.

Don't confuse: SQL = fixed schema + JOINS + ACID (correctness first). NoSQL = flexible schema + horizontal scale (scale first). Neither is 'better', they trade correctness ease for scale ease.

8. Database Replication (Data)

Analogy: Ek hi ledger ho aur 100 log padhna chahein toh line lag jaaye. Toh ledger ki kai photocopy bana do (replicas). Likhna sirf asli ledger pe (master), par padhna kisi bhi copy se (read replicas). Agar asli ledger jal jaaye, ek copy ko asli bana do.

What it does: Keeps multiple copies of the database in sync so reads can be spread across copies and the system survives a server dying.

Spot it when you hear:

- 'How do you handle many more reads than writes?'
- 'What if the database server crashes?'
- 'How do you make the DB highly available?'

How it works

Writes -> [Master / Primary] -> copies changes to -> Replica 1
-> Replica 2

Reads -> any replica (spread the read load)

Master dies -> promote a replica to be the new master (failover)

Trap: Replication lag. A replica is a few milliseconds behind the master. So a user writes data, then immediately reads from a lagging replica and DOESN'T see their own write. Fixes: read-your-own-writes from the master, or wait for the replica to catch up. This is exactly the 'eventual consistency' problem in action.

Real use: Read-heavy systems (most of them). master-slave (one writer, many readers) is the common setup; master-master allows two writers but risks write conflicts.

Don't confuse: Replication = COPY the whole DB many times (for reads + safety). Sharding = SPLIT the data into pieces (for write scale + size). You often do both: shard, then replicate each shard.

9. Sharding / Partitioning (Data)

Analogy: Ek hi sheher ka phonebook itna mota ho gaya ki ek almari mein nahi aata. Toh use do hisson mein todo: A se M ek volume, N se Z doosra. Ab dono almari alag, dono halki. Sharding wahi hai: ek bahut badi table ko tukdon (shards) mein todna, har tukda alag machine pe.

What it does: Splits one huge dataset across many machines by a shard key, so each machine holds and serves only a slice.

Spot it when you hear:

- 'The data is too big for one machine'

- 'Writes are the bottleneck, not reads'
- 'How do you scale the database beyond one box?'

How it works

`user_id % 4` decides the shard:

shard 0 (ids ...0,4,8) shard 1 (...1,5,9) shard 2 shard 3

Each shard = its own machine, holding 1/4 of the data and 1/4 of the writes. Pick a shard key that spreads load EVENLY (user_id good, country bad).

Trap: A bad shard key creates hotspots. Shard by country and India's shard melts while a tiny-country shard idles. The 'celebrity problem': shard a social app by user and one celebrity's shard gets hammered. Also cross-shard JOINS and re-sharding are painful, which is why consistent hashing (next block) exists.

Real use: Any database that outgrows a single server. The shard key choice is the single most important decision and a classic interview probe.

Don't confuse: Sharding SPLITS data across machines (scale write + size). Replication COPIES the same data (scale read + survive failure). Sharding = different data per box; replication = same data per box.

10. Database Indexing (Data)

Analogy: Kitaab mein ek topic dhoondhna ho toh tum har page nahi palatte, peeche index dekhte ho: 'topic X -> page 84'. Database index bhi wahi hai. Bina index ke DB poori table scan karta hai (har page palatna). Index ke saath wo seedha row tak jaata hai.

What it does: A sorted lookup structure on a column that turns a slow full-table scan into a fast direct jump to the rows you want.

Spot it when you hear:

- 'This query is slow'
- 'How do you speed up searches on a column?'
- 'Why is WHERE email = ... slow on a big table?'

How it works

No index: `SELECT * WHERE email=x` -> scan all 10,000,000 rows (slow)

With index on email: jump straight to the matching rows (fast)

Trade-off: indexes speed up READS but slow down WRITES (the index must update too).

Trap: Indexing every column 'to be safe' backfires: every insert/update now has to update all those indexes, so writes get slower and storage grows. Index the columns you actually filter or sort by (WHERE, JOIN, ORDER BY), not all of them.

Real use: The first thing you reach for when a query is slow. Built on B-trees (range queries) or hash indexes (exact match).

Don't confuse: An index speeds up reads INSIDE one database (find rows fast). A cache speeds up reads by avoiding the database entirely (keep a copy in RAM). Index = faster DB; cache = skip the DB.

11. CAP Theorem (Data)

Analogy: Ek distributed system mein teen cheezein chahiye: Consistency (sab ko same latest data dikhe), Availability (system hamesha jawaab de), Partition tolerance (network toot jaaye toh bhi chale). CAP kehta hai: jab network toot-ta hai, tum C aur A dono nahi paa sakte. Ek chunna padega.

What it does: In a distributed system, when the network between nodes breaks, you must choose between staying Consistent (refuse stale answers) or staying Available (answer with possibly-stale data).

Spot it when you hear:

- 'Consistency or availability, which matters more here?'
- 'What happens during a network partition?'
- 'Can the system show slightly old data?'

How it works

Network is fine -> you get all three, no problem.

Network PARTITION (nodes can't talk) -> pick one:

CP: refuse to answer rather than give wrong data	(banking, payments)
AP: keep answering, accept it might be stale	(feeds, likes, DNS)

Trap: CAP is only about the moment of a network partition, not normal operation. And 'partition tolerance' is not optional in a real distributed system (networks DO fail), so the real choice is just CP vs AP. Saying 'I'll pick CA' shows you misunderstood it.

Real use: The frame for every 'strong vs eventual consistency' decision. Banking leans CP; social feeds, shopping carts, DNS lean AP.

Don't confuse: CAP is the THEORY (pick C or A during a partition). Consistency models (next block) are the PRACTICE (how stale, exactly?). CAP says you must choose; consistency models describe what you chose.

12. Consistency Models (Data)

Analogy: Strong consistency: jaise hi message bheja, har device pe turant wahi latest dikhe, guarantee. Eventual consistency: message bheja, abhi kisi friend ke phone pe thodi der baad pahunchega, par 'eventually' sab ke paas pahunch jaayega. WhatsApp ke likes/counts eventual hote hain, bank balance strong hota hai.

What it does: Strong = every read sees the latest write, always. Eventual = reads may briefly see old data, but all copies converge soon.

Spot it when you hear:

- 'Can users tolerate seeing slightly stale data?'
- 'Does every read need the absolute latest value?'
- 'Like counts vs account balance'

How it works

STRONG: write -> every reader instantly sees the new value (slower, more coordination)

use for: money, inventory count, seat booking

EVENTUAL: write -> some readers see old value for a moment -> all catch up (fast, scalable)

use for: likes, view counts, feeds, DNS, social follows

Trap: Demanding strong consistency everywhere kills scale and availability (it needs coordination on every read). Most consumer features are perfectly fine eventual: nobody is

harmed if a like count is off by 2 for one second. Match the model to how much staleness the feature can actually tolerate.

Real use: The practical output of a CAP decision. Pick per-feature, not per-system: a shopping app can be eventual for the catalog and strong for checkout.

Don't confuse: Strong = correctness first (slower). Eventual = speed/scale first (briefly stale). This is the SAME C-vs-A tension from CAP, just applied feature by feature.

13. Consistent Hashing (Distribution & Comms)

Analogy: Maan lo 4 servers gol mez pe baithe hain. Data ko mez ke around ek ghadi ki tarah rakho, har item ghadi-wise apne agle server ko milta hai. Ab ek naya server add karo: sirf uske aas-paas ka thoda data hilta hai, baaki sab apni jagah. Simple ' $id \% N$ ' mein N badalte hi POORA data reshuffle ho jaata hai, ye usse bachata hai.

What it does: A hashing scheme that lets you add or remove servers while moving only a small fraction of the data, instead of remapping everything.

Spot it when you hear:

- 'How do you add/remove cache or DB nodes without remapping everything?'
- 'Avoid a massive reshuffle when the cluster size changes'
- 'How does a distributed cache decide which node holds a key?'

How it works

Plain $id \% N$: add ONE server ($N: 4 \rightarrow 5$) $\rightarrow$ nearly ALL keys move (disaster)

Consistent hashing: servers + keys placed on a ring ($0..2^{32}$)

each key $\rightarrow$ next server clockwise

add a server $\rightarrow$ only the keys between it and its neighbor move ($\sim 1/N$ of data)

'virtual nodes' spread each server to many ring spots for even load

Trap: Without virtual nodes, a few servers can land close together on the ring and get uneven load. Real systems give each physical server many virtual positions on the ring to even it out. This is the one mechanism behind it that interviewers like to hear.

Real use: Distributed caches (Memcached clients), Cassandra, DynamoDB partitioning, sharded systems that resize often.

Don't confuse: Plain sharding ($\text{id} \% N$) is simple but breaks badly when N changes. Consistent hashing is sharding done RIGHT for clusters that grow and shrink, only $\sim 1/N$ of keys move on a resize.

14. Message Queue (Distribution & Comms)

Analogy: Restaurant mein order do, turant token milta hai, tum free. Kitchen orders ko ek line mein leke apne pace pe banati hai. Message queue wahi token system hai: ek service kaam ko queue mein daal deti hai (producer), doosri service use baad mein uthati hai (consumer). Dono ek doosre ka intezaar nahi karte.

What it does: A buffer between services that lets a producer drop a task and move on, while a consumer processes it later, decoupling them.

Spot it when you hear:

- 'This task is slow, the user shouldn't wait for it'
- 'Decouple two services / smooth out traffic spikes'
- 'Send emails / process videos / update analytics in the background'

How it works

User uploads video -> API drops a 'process this' message -> returns instantly
[Queue: msg1, msg2, msg3 ...]

Worker pulls msg -> transcodes video -> marks done (happens async)

Spike of 10,000 uploads? The queue buffers them; workers drain at their own pace.

Trap: Messages can be delivered more than once, so consumers must be idempotent (processing the same message twice does no extra harm). Also decide on ordering needs and what happens to messages that keep failing (a dead-letter queue catches them).

Real use: Email/SMS sending, video transcoding, payment processing, analytics, order pipelines. Kafka (high-throughput log), RabbitMQ, AWS SQS.

Worth knowing: Two flavors: a QUEUE (one message goes to one worker, like task processing) and PUB/SUB (one message fans out to many subscribers, like notifications). Kafka does both.

Don't confuse: A message queue makes work ASYNC (drop it now, done later). A load balancer routes SYNC requests (caller waits for the answer). Queue = fire and forget; LB = wait for the reply.

15. Microservices vs Monolith (Distribution & Comms)

Analogy: Monolith ek bada general store hai: sab kuch ek hi building mein, ek hi team chalati hai, simple par badhne pe bheed. **Microservices** ek mall hai jisme alag-alag specialty shops (orders, payments, search), har shop apni team, apna shutter. **Flexible** par mall manage karna (network, coordination) mushkil.

What it does: Monolith = one codebase/deployable doing everything. Microservices = many small independent services, each owning one job and deployed separately.

Spot it when you hear:

- 'Should this be one service or many?'
- 'How do teams deploy independently?'
- 'When does a monolith stop scaling?'

How it works

MONOLITH: [one app: orders + payments + search + users] one deploy
start here. simple to build, test, deploy. one DB.

MICROSERVICES: [orders] [payments] [search] [users] each its own deploy + DB
scale + ship independently, but network calls, more ops, harder debugging.

Trap: Starting a brand-new project with microservices is a classic over-engineering trap. You pay all the costs (network latency, distributed debugging, data consistency across services) before you have the scale that justifies them. The honest answer in most interviews: 'start with a monolith, split out services when a part actually needs to scale or ship on its own'.

Real use: Monolith for startups / MVPs / small teams. Microservices for large orgs with many teams and parts that scale very differently (e.g. search vs billing).

Don't confuse: Monolith = one unit (simple, but everything scales and ships together).

Microservices = many units (independent scale + deploy, but distributed-system pain). Neither is 'modern = better'; it's a team-size and scale call.

8. The confusion-killer - the pairs people always mix up

This table exists because these pairs are where students lose the interview. Memorize the tell in the right column.

The pairs people always mix up

This is where the interview is won or lost. Memorize the tell.

Replication COPY the same data many times (read scale + safety).	vs	Sharding SPLIT data across machines (write scale + size).
Strong consistency Every read sees the latest write (slower).	vs	Eventual consistency Reads can be briefly stale, copies catch up (faster).
Cache Fast copy in RAM near your server.	vs	CDN Static-file copy placed near the USER worldwide.
Load balancer SPREADS traffic across copies of one service.	vs	API gateway Smart front door: auth + routing + rate limits.
Horizontal scaling MORE machines (needs stateless + a load balancer).	vs	Vertical scaling ONE bigger machine (simple, but has a ceiling).

The classic mix-ups. Left vs right - the tell is all you need.

Often confused	The one tell that separates them
Horizontal vs Vertical scaling	Horizontal = MORE machines (needs stateless + a load balancer). Vertical = ONE bigger machine (has a ceiling).
Replication vs Sharding	Replication = COPY the same data many times (read scale + safety). Sharding = SPLIT data across machines (write scale + size).
SQL vs NoSQL	SQL = fixed schema + JOINS + ACID (correctness easy). NoSQL = flexible schema + scales out (scale easy).
Strong vs Eventual consistency	Strong = every read sees the latest write (slower). Eventual = reads can be briefly stale, all copies catch up (faster).
Cache vs CDN	Cache = fast copy in RAM near your server. CDN = copy of static files placed near the USER worldwide.
Load balancer vs API gateway	Load balancer SPREADS traffic across copies of one service. Gateway is the smart front door (auth + routing + limits).

Often confused	The one tell that separates them
Message queue vs Load balancer	Queue = async (drop work now, done later). Load balancer = sync (caller waits for the reply).
L4 vs L7 load balancing	L4 routes on IP/port (fast, dumb). L7 reads the HTTP request so it can route by URL or cookie (smarter).

9. Back-of-the-envelope numbers every fresher must know

Interviewers love 'roughly how many servers / how much storage?' You are NOT expected to be exact. You ARE expected to reason in round numbers and orders of magnitude. Memorize these few anchors and you can estimate almost anything.

Powers of two (data sizes)

Power	Approx value	Means
2^{10}	~1 thousand	1 KB
2^{20}	~1 million	1 MB
2^{30}	~1 billion	1 GB
2^{40}	~1 trillion	1 TB
2^{50}	~1 quadrillion	1 PB

Time: requests per second

One day is about 86,400 seconds, round it to ~100,000 (10^5) for fast mental math.

Daily requests	Rough QPS (per second)	How to get it
1 million / day	~12 QPS	$1,000,000 / 86,400$
10 million / day	~115 QPS	$10,000,000 / 86,400$
100 million / day	~1,160 QPS	$100,000,000 / 86,400$
1 billion / day	~11,600 QPS	$1,000,000,000 / 86,400$

Peak traffic is usually 2x to 3x the average, so multiply your average QPS by ~2-3 to size for the busy hour. And reads usually dwarf writes: a 100:1 read-to-write ratio is a common, safe assumption to state out loud unless told otherwise.

Latency: orders of magnitude (memorize the ratios, not exact ns)

First, the units. Every latency number is just a power of ten. Lock this ladder in your head and the rest is easy.

Unit	Power of ten	In seconds	How to think of it
second (s)	10^0	1	A slow human blink takes a few hundred ms; one second is forever to a CPU.
millisecond (ms)	10^{-3}	1 / 1,000	1 ms = 1,000 microseconds. Disk and network live here.
microsecond (us)	10^{-6}	1 / 1,000,000	1 us = 1,000 nanoseconds. RAM reads and datacenter hops live here.
nanosecond (ns)	10^{-9}	1 / 1,000,000,000	1 ns = one billionth of a second. CPU cache lives here.

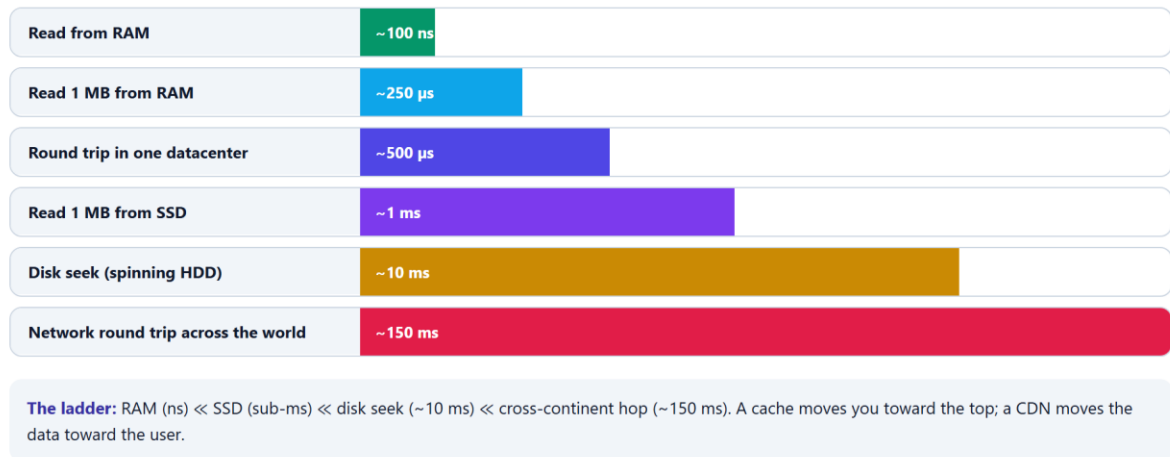
YOU HAVE TO REMEMBER THIS: 1 ms = 1,000 us = 1,000,000 ns. ns < us < ms < s, each step is 1000x.

Now the actual numbers. These are the famous 'latency numbers every programmer should know' (Jeff Dean). You don't need them exact, but knowing the rough scale of each is what lets you say 'that read is a disk seek, so ~10 ms' on the spot.

Operation	Time	In ns
L1 cache reference	0.5 ns	0.5
Branch mispredict	5 ns	5
L2 cache reference	7 ns	7
Mutex lock / unlock	25 ns	25
Main memory (RAM) reference	100 ns	100
Compress 1 KB with Zippy	3 us	3,000
Send 2 KB over 1 Gbps network	20 us	20,000
SSD random read	150 us	150,000
Read 1 MB sequentially from RAM	250 us	250,000
Round trip in the same datacenter	0.5 ms	500,000
Read 1 MB sequentially from SSD	1 ms	1,000,000
Disk seek (spinning HDD)	10 ms	10,000,000
Read 1 MB sequentially from disk	20 ms	20,000,000
Network round trip across the world	~150 ms	150,000,000

The latency ladder - memorize the ratios, not the exact numbers

Every caching and CDN decision is just moving data UP this ladder, closer to RAM and closer to the user.



The latency ladder. Caching and CDNs are just ways to move data up this ladder.

The one ladder to remember: RAM (nanoseconds) is far faster than SSD (sub-millisecond), which is faster than disk seek (~10 ms), which is faster than a cross-continent network hop (~150 ms). Every caching and CDN decision is just moving data UP this ladder, closer to RAM and closer to the user.

Availability: the 'nines' (what an SLA really promises)

When someone says '99.9% uptime', this is the downtime budget it actually buys. Each extra nine is roughly 10x less downtime and a lot more engineering.

Availability	Name	Downtime per year	Downtime per day
99%	two nines	~3.65 days	~14 minutes
99.9%	three nines	~8.8 hours	~1.4 minutes
99.99%	four nines	~52 minutes	~8.6 seconds
99.999%	five nines	~5.3 minutes	~0.86 seconds

Two facts interviewers like: components in SERIES multiply (two 99.9% services in a row = 99.8% together, worse), while REDUNDANT copies in parallel add nines (two 99.9% copies = 99.9999%). That is the whole math case for redundancy.

Latency percentiles - p50, p95, p99 (don't say 'average')

Interviewers ask 'what's your p99?'. Latency is reported as percentiles, not an average, because a few slow requests get hidden by an average but still hurt real users.

Term	Means	Why it matters
p50 (median)	Half of requests are faster than this.	The typical experience, but it hides the slow tail.
p95	95% of requests are faster; the slowest 5% are worse.	Starts to show the users who are having a bad time.
p99	99% are faster; the slowest 1% are worse.	The tail you actually optimize. At scale, 1% is millions of requests.

Say 'I'd track p99 latency, not the average' and you instantly sound senior. Averages lie; tail latency is what users feel and what SLOs are written against.

A worked estimate (do it like this on the whiteboard)

'Design a URL shortener for 100 million new links a day.' Writes: $100M / 86,400 \sim 1,160$ writes per second. Reads at 100:1 $\sim 116,000$ reads per second (so this is read-heavy: reach for caching + replicas). Storage: each link row ~ 500 bytes, so $100M * 500$ bytes = 50 GB per day, about 18 TB per year. State these three numbers out loud and you've already shown the interviewer how you think.

10. The framework - attack ANY system design problem in 7 steps

The biggest fear is 'where do I even start?'. You start by NOT jumping to a diagram. Walk these 7 steps in order, out loud, every single time. The structure is what scores points; the perfect architecture is secondary.

11. Clarify requirements (2-3 min). Pin down functional needs ('shorten a URL, redirect it') and non-functional needs (scale, latency, consistency). Ask before you assume.
12. Estimate scale. Daily users, QPS, read:write ratio, storage per year. Use the section-9 anchors. This decides everything downstream.
13. Define the API. The 2-3 core endpoints (e.g. POST /shorten, GET /{code}). Keeps the design concrete and honest.
14. Sketch the high-level diagram. Client -> LB -> servers -> cache -> DB, plus a CDN and a queue if needed. This is the request-lifecycle spine from section 3.
15. Design the data model. What tables/collections, what's the key, SQL or NoSQL, and why.
16. Scale the design. Now add the building blocks where the bottleneck is: caching for read load, replication + sharding for the DB, a queue for slow work, a CDN for static files.

17. Discuss trade-offs + failures. What breaks if a server dies, where's the single point of failure, strong vs eventual consistency for each part. THIS is where senior signal shows.

Mnemonic: Requirements, Estimate, API, Diagram, Data, Scale, Trade-offs. The first two steps (clarify + estimate) are where most freshers rush and lose points. Slow down there. An interviewer would rather see a clear small design with honest trade-offs than a giant diagram you can't defend.

11. Interview triggers - if they say X, reach for Y

If the interviewer says...	...reach for
'How does this handle more traffic?'	Horizontal scaling + load balancer
'This read is slow / DB is hammered'	Caching (Redis), then read replicas
'Users are global and assets are slow'	CDN for static content
'The data is too big for one machine'	Sharding (partition by a good key)
'What if the database server dies?'	Replication + failover to a replica
'This query on a big table is slow'	Add an index on the filtered column
'This task is slow, don't make the user wait'	Message queue + background workers
'Stop one user from spamming the API'	Rate limiting (token bucket)
'Can users tolerate slightly stale data?'	Eventual consistency (faster, scales)
'This must always show the latest value'	Strong consistency (money, inventory)
'Many services, one entry point'	API gateway
'Add/remove nodes without remapping all keys'	Consistent hashing

From problem statement to building block - 3 worked examples

The hard part is not naming a block. It is hearing it inside the interviewer's words. Here is the leap, three times.

1. 'Our timeline takes 4 seconds to load because we query 5 tables on every open.' -> same data read again and again -> Caching. Cache the assembled timeline in Redis with a short TTL; rebuild on a miss.

2. 'Sending the confirmation email makes checkout feel slow.' -> slow work the user shouldn't wait for -> Message queue. Drop an 'email' message on the queue, return success instantly, let a worker send it.

3. 'Our single Postgres box can't take the write volume anymore.' -> writes outgrew one machine -> Sharding. Split rows across machines by a key that spreads load evenly (user_id, not country).

12. Common mistakes in HLD rounds

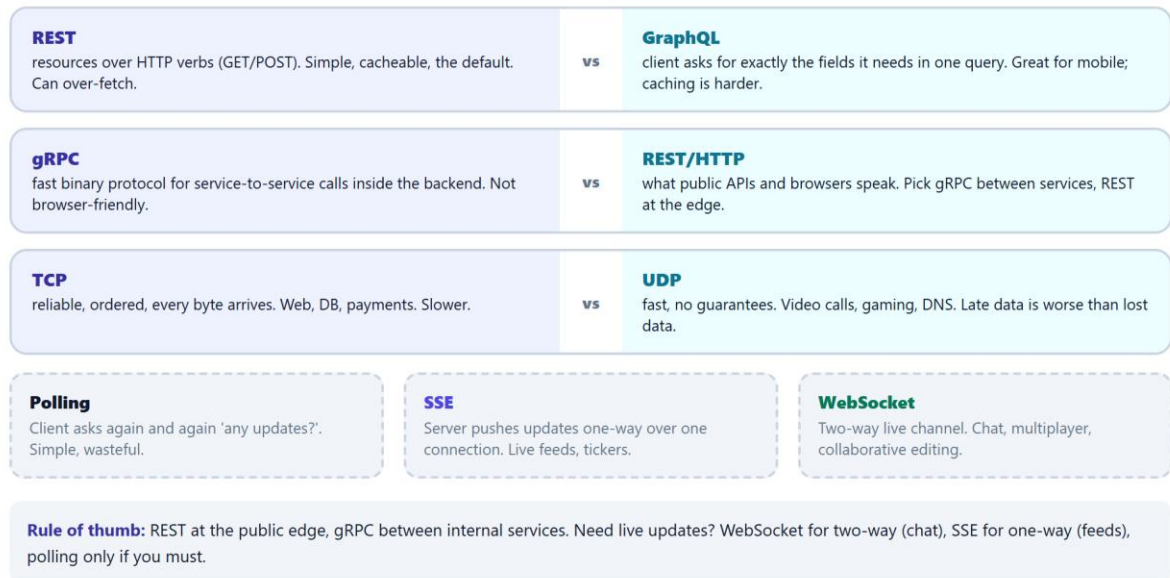
Mistake	Why it loses points	Do this instead
Jumping straight to a diagram	You skip requirements and scale, so the design solves the wrong problem.	Clarify requirements and estimate scale FIRST (steps 1-2). Slow down there.
Adding every buzzword	Kafka + microservices + sharding on a problem with 100 users screams 'I memorized words'.	Start simple. Add a building block only when you've named the bottleneck it fixes.
Microservices for a brand-new app	You pay distributed-system costs before you have the scale to justify them.	Start with a monolith. Split out a service only when a part truly needs independent scale.
Never mentioning trade-offs	Picking a tool without naming what you give up reads as shallow.	Every choice: name what you trade away ('cache -> lower latency, risk of stale data').
Ignoring failures	Real systems fail. No failure talk = no senior signal.	Call out single points of failure and how you'd survive a server/DB dying.
Strong consistency everywhere	It kills scale and availability when most features don't need it.	Use eventual consistency where staleness is harmless (likes, feeds); strong only for money/inventory.
Staying silent while thinking	The interviewer can't read your mind; silence reads as stuck.	Narrate. 'I'm choosing NoSQL here because the schema varies and we need write scale.'

13. Communication & APIs - how clients and services talk

Once you have boxes on the whiteboard, the interviewer asks how they talk. You don't need deep protocol knowledge, just the right default for each situation and why.

How clients and services talk

The communication layer. Know the protocol, the API style, and how to get real-time updates.



The communication layer. Pick the protocol, the API style, and the real-time option that fits.

API styles - how a client asks for data

Style	Use it when	The catch
REST	The default for public APIs. Resources over HTTP verbs (GET, POST, PUT, DELETE). Simple, cacheable.	Can over-fetch (you get fields you don't need) or need several round trips.
GraphQL	The client needs exactly certain fields, often on mobile. One query, no over-fetching.	Caching is harder and it shifts complexity to the server.
gRPC	Fast service-to-service calls INSIDE your backend. Binary, efficient, supports streaming.	Not browser-friendly, so you rarely expose it to the public.

Safe default answer: REST at the public edge, gRPC between internal microservices. GraphQL when one screen pulls from many sources and you want one round trip.

Protocols + real-time - the one-liners

- TCP vs UDP: TCP is reliable and ordered (web, DB, payments). UDP is fast with no guarantees (video calls, gaming, DNS) where late data is worse than lost data.
- HTTP status codes you should name: 200 OK, 201 Created, 301 redirect, 400 bad request, 401 unauthorized, 404 not found, 429 too many requests (rate limited), 500 server error.

- Real-time updates: polling (client asks again and again, wasteful), SSE (server pushes one-way, good for live feeds), WebSocket (two-way live channel, good for chat and multiplayer).
- Webhook: the reverse of polling. Instead of you asking, the other service calls YOUR URL when an event happens (a payment succeeds, a build finishes).

14. Interview FAQs - the questions every panel circles back to

These come up in almost every HLD round, usually as quick follow-ups to check you actually understand the block you just named. Short, confident answers win here.

They ask	You say (crisp)
Latency vs throughput?	Latency = time for one request. Throughput = requests handled per second. Batching raises throughput but adds latency, so you trade one for the other.
When NoSQL over SQL?	When the schema is flexible or changing, you need horizontal write scale, and access is mostly simple key lookups, not multi-table JOINS or transactions.
How do you scale reads?	Cache the hot data (Redis), then add read replicas and send reads to them. Reads are usually far more frequent than writes, so this covers most load.
How do you scale writes?	Shard the data across machines by a key that spreads load evenly, and push slow writes through a message queue so they don't block the user.
What is a single point of failure?	Any one box whose death takes the whole system down. Remove it by running two or more of everything: redundant load balancers, replicas, multiple zones.
Why keep web servers stateless?	So the load balancer can send any request to any server. Session and state go into a shared store (Redis) or the DB, never local server memory.
How do you avoid stale cache data?	Set a TTL so entries auto-expire, or invalidate the key on every write. Pick based on how much staleness the feature can tolerate.
How do you pick a shard key?	High cardinality, spreads load evenly (user_id good, country bad). Avoid keys that create hotspots, like sharding a social app by a celebrity user.
Strong vs eventual consistency, with an example?	Strong: a bank balance, must always be exact. Eventual: a like or follower count can be off by a few for a second and nobody is harmed.
What does CAP actually force you to choose?	Only during a network partition: stay Consistent (refuse stale answers) or stay Available (answer with maybe-stale data). Partition tolerance is not optional.
How do you handle a sudden 10x spike?	Autoscale stateless servers behind the LB, lean on cache and CDN, and buffer write bursts in a message queue so workers drain at their own pace.
How do you make a system highly available?	Redundancy and failover everywhere, health checks to detect dead nodes, replicas ready to promote, and spreading across availability zones.
When does a message queue help?	When work is slow and the user shouldn't wait: emails, video processing, analytics. It also absorbs traffic spikes and decouples services.
How do you speed up a slow query?	Add an index on the filtered or sorted column first. Then consider caching the result, or denormalizing if expensive JOINS are the cost.
First thing you do in a design round?	Clarify requirements and estimate scale (QPS, storage). Never jump to a diagram first; the numbers decide the whole design.
How would you reduce database load?	Cache hot reads, route reads to replicas, add indexes, and move heavy writes to a queue. The DB should be the last resort, not the first stop.

Rapid-fire one-liners (know these cold)

- Read-heavy -> cache + replicas. Write-heavy -> shard + queue.
- Static files -> CDN. Hot data -> Redis. Big files -> object storage (S3).
- Need exact data -> strong consistency. Can tolerate stale -> eventual.
- One bigger box -> vertical (has a ceiling). More boxes -> horizontal (needs stateless + LB).
- Protect an API -> rate limiting. One front door for many services -> API gateway.
- Resize a cluster without remapping everything -> consistent hashing.

15. More concepts to recognize - know the word, say the one-liner

The 15 blocks above cover almost every fresher round. These extra terms come up in follow-ups. You rarely have to design them cold, but recognizing the word and giving the one-liner shows depth. Know the hook, don't stress the implementation.

Concept	One line to remember	What it does
Reverse proxy	Servers ke saamne ek chehra. (hides the backend)	Sits in front of servers to hide them, cache, compress, and terminate SSL. Nginx is the classic one.
Bloom filter	'Pakka nahi hai' jaldi bata de. (fast maybe-check)	A tiny memory structure that says 'definitely not present' or 'maybe present' fast, to skip pointless DB lookups.
WebSocket / SSE / polling	Server khud push kare vs tum baar-baar poocho. (live updates)	Ways to get real-time updates: polling (ask repeatedly), SSE (server pushes one-way), WebSocket (two-way live channel).
Idempotency	Do baar chala, ek hi asar. (safe retries)	An operation you can safely repeat with no extra effect. Critical for retries and exactly-once-feeling payments.
Circuit breaker	Baar-baar fail ho raha, kuch der ke liye band. (fuse)	Stops calling a failing service for a while so one dead service doesn't drag down the whole system.
Leader election	Ek ko boss chuno, baaki maanein. (one coordinator)	How a cluster agrees on one 'leader' node to coordinate, and picks a new one if it dies. ZooKeeper, Raft.
Blob / object storage	Badi files ke liye alag godown. (S3)	Cheap storage for large files (images, video, backups) outside the database. AWS S3 is the standard.
Write-ahead log (WAL)	Pehle likho diary mein, phir karo. (durability)	Record the change to a log BEFORE applying it, so nothing is lost if the system crashes mid-write.
Heartbeat / health check	Zinda ho? haan/naa. (is it alive)	Periodic 'are you alive?' pings so the system detects and routes around dead nodes.
Quorum	Bahumat se faisla. (majority agrees)	Require a majority of replicas to agree on a read/write, balancing consistency and availability.
Containers + orchestration	App ko dabbe mein pack karo, manager chalaye. (Docker + Kubernetes)	A container (Docker) packs an app with its dependencies so it runs anywhere; Kubernetes runs and scales thousands of them.

Concept	One line to remember	What it does
Serverless	Server tum mat sambhalo, code chadhaao. (AWS Lambda)	You ship just functions; the cloud runs and scales them per request. Great for spiky, event-driven work.
Event-driven architecture	Kuch hua toh sab ko event bhej do. (Kafka events)	Services react to events on a bus instead of calling each other directly. Loose coupling, easy to add consumers.
Service discovery	Service kahan chal rahi hai, registry bataye. (Consul / etcd)	A registry that tracks where each service instance lives, so callers find them even as instances come and go.
SLA / SLO / SLI	Promise, target, aur naap. (99.9% uptime)	SLA = the promise to users, SLO = the internal target, SLI = the actual measured number (e.g. uptime, latency).
Saga / 2-phase commit	Kai DB pe ek transaction, undo ke saath. (distributed transaction)	Ways to keep a transaction correct across multiple services: 2PC locks then commits; a saga does steps with compensating undos.
Geohashing / quadtree	Map ko grid mein todo, paas waale dhoondo. (Uber, Yelp)	Spatial indexes that turn lat/long into searchable cells, so 'find drivers near me' is fast. Used in location apps.
Change data capture (CDC)	DB mein jo badla, stream pe bhej do. (Debezium)	Streams every database change as an event so caches, search indexes, and analytics stay in sync without dual writes.
Reliability vs fault tolerance	Bharosa vs gir ke bhi chalna. (keeps working)	Reliability = it does the right thing consistently; fault tolerance = it keeps serving even when a component dies.
Distributed locking	Ek hi banda ek time pe likhe. (Redis lock, ZooKeeper)	A lock shared across machines so only one process touches a resource at a time, avoiding race conditions.
Observability (logs/metrics/traces)	System andar se dikhe. (Grafana, Jaeger)	The three pillars: logs (what happened), metrics (numbers over time), traces (one request across services). How you debug at scale.
Authn vs Authz	Tum kaun ho vs tumhe kya allowed hai. (login vs permissions)	Authentication = proving who you are (login). Authorization = what you're allowed to do (permissions). Different steps, often confused.
OAuth + JWT	'Google se login' aur ek signed token. (delegated access)	OAuth lets an app act on your behalf without your password ('login with Google'); a JWT is the signed token that carries your identity.
TLS / encryption	Taala in transit aur at rest. (HTTPS)	TLS encrypts data moving over the network (the S in HTTPS); also encrypt data at rest in the DB. The baseline of any secure system.
Vector database	Meaning se search, keyword se nahi. (AI / RAG)	Stores embeddings and finds nearest-neighbor matches by meaning. Powers semantic search and AI/RAG apps. Pinecone, pgvector.

If the core 15 are solid, these 10 are pure bonus. In a fresher round you'll almost never design one. Recognize the word, say the one-liner, move on.

16. Revision - the last 20 minutes before the round

The 15 analogies, one line each

- Load Balancer - station ki kai ticket windows, bheed ko baant do.
- Horizontal vs Vertical - zyada cooks vs ek super-chef.
- Caching - important formulae chit pe desk pe, library baar-baar nahi.
- CDN - door ki factory nahi, paas ki kirana dukaan.
- Rate Limiting - club bouncer per-minute entry ginta hai.
- API Gateway - building ka reception, sab ek door se.
- SQL vs NoSQL - pakka ledger vs lachila sticky-note box.
- Replication - ledger ki photocopy, sab padh sakein.
- Sharding - mota phonebook A-M, N-Z mein todo.
- Indexing - kitaab ke peeche ka index, page seedha milta hai.
- CAP - teen mein se do, partition pe C ya A chuno.
- Consistency - turant pakka (bank) vs thodi der mein pakka (likes).
- Consistent Hashing - gol mez, naya mehmaan sabko na hilaaye.
- Message Queue - order token lo, kitchen apne pace pe banaye.
- Microservices vs Monolith - specialty shops ka mall vs ek bada general store.

The request spine you draw first, every time

Client -> DNS -> Load Balancer -> Web Server -> Cache -> Database. CDN off to the side for static files. Message queue off to the side for slow background work. Start here, then add boxes where the bottleneck is.

The 7-step framework in one breath

Requirements -> Estimate scale -> API -> Diagram -> Data model -> Scale it -> Trade-offs. Slow down on the first two. Narrate everything.

The 4 confusion tells to re-read right before going in

- Replication = copy the same data. Sharding = split data into pieces.
- Strong consistency = always latest (money). Eventual = briefly stale, then catches up (likes).
- Cache = fast copy in RAM near your server. CDN = static-file copy near the user.
- Horizontal = more machines (stateless + LB). Vertical = one bigger machine (has a ceiling).

You learned 15 building blocks as 15 pictures, not 15 definitions. The interviewer will describe

a problem, not name a block. Match the situation to a picture, draw the request spine, name your trade-offs out loud. You don't need to design Google. You need to think clearly and talk. You've practised this. Go.

@rathoreadiitya